

Quickhull

Quickhull is a method of computing the convex hull of a finite set of points in n -dimensional space. It uses a divide and conquer approach similar to that of quicksort, from which its name derives. Its worst case time complexity for 2-dimensional and 3-dimensional space is $O(n^2)$, but when the input precision is restricted to $O(\log n)$ bits, its worst case time complexity is conjectured to be $O(n \log r)$, where n is the number of input points and r is the number of processed points (up to n).^[1]

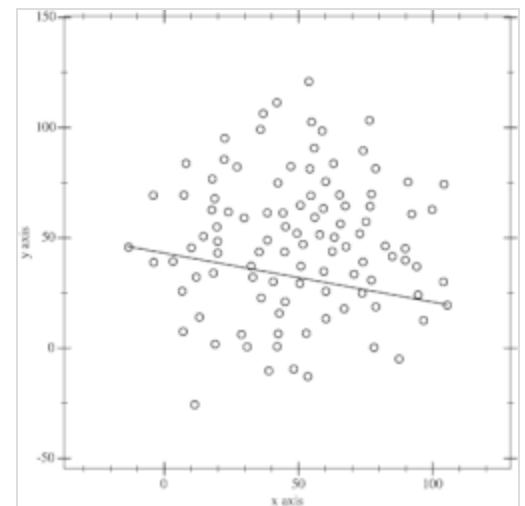
N-dimensional Quickhull was invented in 1996 by C. Bradford Barber, David P. Dobkin, and Hannu Huhdanpaa.^[1] It was an extension of Jonathan Scott Greenfield's 1990 planar Quickhull algorithm, although the 1996 authors did not know of his methods.^[2] Instead, Barber et al. describe it as a deterministic variant of Clarkson and Shor's 1989 algorithm.^[1]

Algorithm

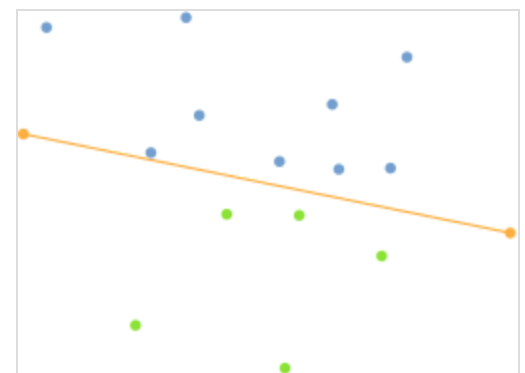
The 2-dimensional algorithm can be broken down into the following steps:^[2]

1. Find the points with minimum and maximum x coordinates, as these will always be part of the convex hull. If many points with the same minimum/maximum x exist, use the ones with the minimum/maximum y, respectively.
2. Use the line formed by the two points to divide the set into two subsets of points, which will be processed recursively. We next describe how to determine the part of the hull above the line; the part of the hull below the line can be determined similarly.
3. Determine the point above the line with the maximum distance from the line. This point forms a triangle with the two points on the line.
4. The points lying inside of that triangle cannot be part of the convex hull and can therefore be ignored in the next steps.
5. Recursively repeat the previous two steps on the two lines formed by the two new sides of the triangle.
6. Continue until no more points are left, the recursion has come to an end and the points selected constitute the convex hull.

The problem is more complex in the higher-dimensional case, as the hull is built from many facets; the data structure needs to account for that and record the line/plane/hyperplane (ridge) shared by neighboring facets too. For d dimensions:^[1]

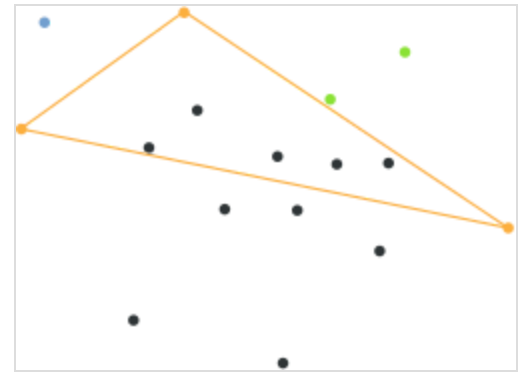


This animation depicts the quickhull algorithm in two dimensions.

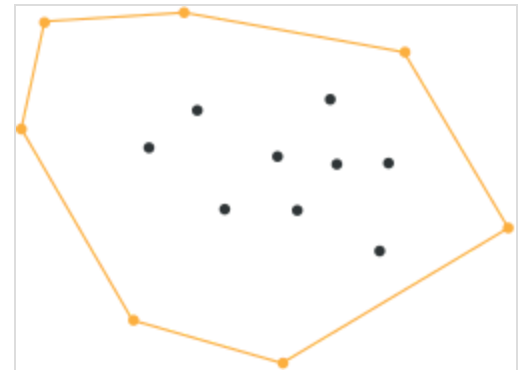


Steps 1-2: Divide the points into two subsets.

1. Pick $d + 1$ points from the set that do not share a plane or a hyperplane. This forms an initial hull with facets $Fs[]$.
2. For each F in $Fs[]$, find all unassigned points that are "above" it; i.e., pointing away from the center of the hull, and assign them to an "outside" set $F.O$ associated with F . The algorithm maintains the invariant that every point that has not been added to the hull but could potentially be a vertex of it is assigned to exactly one outside set.
3. For each F with a non-empty $F.O$:
 1. Find the point p in $F.O$ with the maximum distance from F and add it to the hull. Note that p will not necessarily be a vertex of the final hull, as it might be removed later.
 2. Create a visible set V and initialize it to F . Extend V in all directions for neighboring facets F_v until no further facets are visible from p . F_v being visible from p means that p is above F_v .
 3. The boundary of V then forms the set of horizon ridges H .
 4. Let $F_{new}[]$ be the set of facets created from p and all ridges in H .
 5. Unassign all points in the outside sets of facets in V . For each new facet in $F_{new}[]$, perform step (2) only considering these newly unassigned points to initialize its outside set. Note that every point that remains unassigned at the end of this process lies within the current hull.
 6. Delete the now-internal facets in V from $Fs[]$. Add the new facets in $F_{new}[]$ to $Fs[]$ and continue the iteration.



Steps 3-5: Find a point with the maximum distance, ignore points inside the triangle, and recurse.



Step 6: Recurse until no more points are left.

Pseudocode for 2D set of points

```

Input = a set S of n points
Assume that there are at least 2 points in the input set S of points

function QuickHull(S) is
    // Find convex hull from the set S of n points
    Convex Hull := {}
    Find left and right most points, say A & B, and add A & B to convex hull
    Segment AB divides the remaining (n - 2) points into 2 groups S1 and S2
        where S1 are points in S that are on the right side of the oriented line from A to B,
        and S2 are points in S that are on the right side of the oriented line from B to A
    FindHull(S1, A, B)
    FindHull(S2, B, A)
    Output := Convex Hull
end function

function FindHull(Sk, P, Q) is
    // Find points on convex hull from the set Sk of points
    // that are on the right side of the oriented line from P to Q
    if Sk has no point then
        return
    From the given set of points in Sk, find farthest point, say C, from segment PQ
    Add point C to convex hull at the location between P and Q
    Three points P, Q, and C partition the remaining points of Sk into 3 subsets: S0, S1, and S2
        where S0 are points inside triangle PCQ, S1 are points on the right side of the oriented
        line from P to C, and S2 are points on the right side of the oriented line from C to Q.
  
```

```
FindHull(S1, P, C)
FindHull(S2, C, Q)
end function
```

A pseudocode specialized for the 3D case is available from Jordan Smith. It includes a similar "maximum point" strategy for choosing the starting hull. If these maximum points are degenerate, the whole point cloud is as well.^[3]

See also

- [Convex hull algorithms](#)

References

1. Barber, C. Bradford; Dobkin, David P.; Huhdanpaa, Hannu (1 December 1996). "The quickhull algorithm for convex hulls" (<http://www.cs.princeton.edu/~dpd/Papers/BarberDobkinHuhdanpaa.pdf>) (PDF). *ACM Transactions on Mathematical Software*. **22** (4): 469–483. doi:10.1145/235815.235821 (<https://doi.org/10.1145%2F235815.235821>).
 2. Greenfield, Jonathan S. (1 April 1990). "A Proof for a QuickHull Algorithm" (https://surface.syr.edu/eecs_techreports/65). *Electrical Engineering and Computer Science - Technical Reports*.
 3. Smith, Jordan. "QuickHull 3D" (<https://algolist.ru/maths/geom/convhull/qhull3d.html>). *algolist.ru*. Retrieved 22 October 2019.
- Dave Mount. "Lecture 3: More Convex Hull Algorithms" (<https://web.archive.org/web/20010411073422/http://www.cs.wustl.edu/~pless/506/l3.html>). Archived from the original (<http://www.cs.wustl.edu/~pless/506/l3.html>) on 11 April 2001.

External links

- [Implementing QuickHull \(GDC 2014\) \(http://media.steampowered.com/apps/valve/2014/DirkGregorius_ImplementingQuickHull.pdf\)](http://media.steampowered.com/apps/valve/2014/DirkGregorius_ImplementingQuickHull.pdf) – Algorithm presentation with 3D implementation details.
-

Retrieved from "<https://en.wikipedia.org/w/index.php?title=Quickhull&oldid=1339731952>"